



Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ _____ «Информатика и системы управления»

КАФЕДРА _____ «Теоретическая информатика и компьютерные технологии»

Лабораторная работа № 8
по курсу «Языки и методы программирования»
«Шаблоны в C++. Graph и AVL-дерево»

Студент группы ИУ9-22Б
Булдаков А. С.

Преподаватель
Посевин Д. П.

Москва 2026

Содержание

Цель работы	1
Класс Graph	1
Заголовочный файл graph.hpp	2
Методы класса	4
Алгоритм Прима для MST	4
main()	6
AVL-дерево (дополнительное)	6
Заголовочный файл avl.hpp	7
Вращения	8
Балансировка	9
Вывод	9

Цель работы

Изучить шаблоны классов в C++. Реализовать шаблонный класс Graph для работы с графом и алгоритмом построения минимального остовного дерева (MST).

Дополнительно реализовать AVL-дерево.

Задачи:

- Изучить синтаксис шаблонов в C++.
- Реализовать шаблонный класс Graph с методами add_vertex, add_edge, has_edge, get_edge.
- Реализовать алгоритм Прима для построения MST.
- Реализовать AVL-дерево с балансировкой.

Класс Graph

Шаблонный класс Graph для представления неориентированного графа.

Заголовочный файл graph.hpp

graph.hpp

```

template <typename V, typename T> class Graph {
    struct Edge {
        bool flag;
        T type;
        Edge() : flag(false), type(T()) {}
    };

    int size;
    std::vector<std::vector<Edge>> A;
    std::unordered_map<V, int> vertex_map;

    void ensure_capacity(int v) {
        if (v ≥ A.size()) {
            int newSize = v + 1;
            for (auto &row : A)
                row.resize(newSize);
            A.resize(newSize, std::vector<Edge>(newSize));
        }

        int get_idx(V v) {
            auto it = vertex_map.find(v);
            return (it ≠ vertex_map.end()) ? it→second : -1;
        }

    public:
        Graph() : size(0) {}

        void add_vertex(V v) {
            if (vertex_map.find(v) == vertex_map.end()) {
                int idx = size;
                vertex_map[v] = idx;
                ensure_capacity(idx);
                size++;
            }
        }

        void add_edge(V u, V v, T e) {
            add_vertex(u);
            add_vertex(v);
            int iu = vertex_map[u];
            int iv = vertex_map[v];
            A[iu][iv].flag = true;
            A[iu][iv].type = e;
            A[iv][iu].flag = true;
            A[iv][iu].type = e;
        }

        bool has_edge(V u, V v);

```

Параметры шаблона:

- V — тип вершины
- T — тип веса ребра

Методы класса

graph.hpp

```
bool has_edge(V u, V v) {
    auto it_u = vertex_map.find(u);
    auto it_v = vertex_map.find(v);
    if (it_u == vertex_map.end() || it_v == vertex_map.end())
        return false;
    return A[it_u→second][it_v→second].flag;
}

T get_edge(V u, V v) {
    auto it_u = vertex_map.find(u);
    auto it_v = vertex_map.find(v);
    if (it_u == vertex_map.end() || it_v == vertex_map.end())
        return T();
    return A[it_u→second][it_v→second].type;
}
```

Алгоритм Прима для MST

Минимальное остовное дерево (Minimum Spanning Tree):

graph.hpp

```
std::vector<std::pair<V, V>> mst() {
    if (size == 0) return {};

    std::vector<bool> in_mst(size, false);
    std::vector<T> min_edge(size, std::numeric_limits<T>::max());
    std::vector<int> parent(size, -1);

    min_edge[0] = 0;

    for (int i = 0; i < size; i++) {
        int v = -1;
        for (int j = 0; j < size; j++) {
            if (!in_mst[j] && (v == -1 || min_edge[j] < min_edge[v])) {
                v = j;
            }
        }

        if (min_edge[v] == std::numeric_limits<T>::max()) return {};

        in_mst[v] = true;

        for (int u = 0; u < size; u++) {
            if (!in_mst[u] && A[v][u].flag) {
                T weight = A[v][u].type;
                if (weight < min_edge[u]) {
                    min_edge[u] = weight;
                    parent[u] = v;
                }
            }
        }
    }

    std::vector<std::pair<V, V>> result;
    for (int i = 1; i < size; i++) {
        if (parent[i] != -1) {
            result.push_back({get_vertex(parent[i]), get_vertex(i)});
        }
    }
    return result;
}
```

Алгоритм Прима:

1. starts с первой вершины
2. На каждом шаге выбирает минимальное ребро к непосещенной вершине
3. Добавляет вершину в MST
4. Повторяет пока все вершины не будут посещены

main()

main.cpp

```
int main() {
    Graph<int, int> g;

    g.add_vertex(1);
    g.add_vertex(2);
    g.add_vertex(3);
    g.add_vertex(4);

    g.add_edge(1, 2, 10);
    g.add_edge(1, 3, 15);
    g.add_edge(1, 4, 20);
    g.add_edge(2, 3, 5);
    g.add_edge(3, 4, 8);

    std::cout << "Edge 1-2 exists: " << g.has_edge(1, 2) <<
std::endl;
    std::cout << "Edge 1-4 weight: " << g.get_edge(1, 4) <<
std::endl;

    std::cout << "\nMST edges:" << std::endl;
    auto mst = g.mst();
    for (auto &e : mst) {
        std::cout << e.first << " - " << e.second << std::endl;
    }
}
```

Вывод:

```
Edge 1-2 exists: 1
Edge 1-4 weight: 20
```

MST edges:

```
1 - 2
2 - 3
3 - 4
```

AVL-дерево (дополнительное)

Самобалансирующееся бинарное дерево поиска.

Заголовочный файл avl.hpp

avl.hpp

```
template <typename K, typename V, std::size_t N = 0> class Tree {
private:
    struct Node {
        K key;
        V value;
        int left = -1;
        int right = -1;
        int height = 1;
        Node() = default;
        explicit Node(const K &k) : key(k) {}
    };

    using StorageType =
        std::conditional_t<(N == 0), std::vector<Node>,
        std::array<Node, N>>;

    StorageType storage;
    int root = -1;
    std::size_t node_count = 0;

    int get_height(int node_index) const;
    int get_balance(int node_index) const;
    void update_height(int node_index);
    int rotate_right(int y_index);
    int rotate_left(int x_index);
    int balance_node(int node_index);
    int insert_impl(int node_index, const K &key, int
&target_node_idx);

public:
    Tree();
    V &operator[](const K &key);
    std::size_t size() const noexcept { return node_count; }
};
```

Шаблон с тремя параметрами:

- K — тип ключа
- V — тип значения
- N — размер (0 для динамического)

Вращения

avl.cpp

```
template <typename K, typename V, std::size_t N>
int Tree<K, V, N>::rotate_right(int y_index) {
    int x_index = storage[y_index].left;
    int T2 = storage[x_index].right;

    storage[x_index].right = y_index;
    storage[y_index].left = T2;

    update_height(y_index);
    update_height(x_index);

    return x_index;
}

template <typename K, typename V, std::size_t N>
int Tree<K, V, N>::rotate_left(int x_index) {
    int y_index = storage[x_index].right;
    int T2 = storage[y_index].left;

    storage[y_index].left = x_index;
    storage[x_index].right = T2;

    update_height(x_index);
    update_height(y_index);

    return y_index;
}
```

Балансировка

avl.cpp

```
template <typename K, typename V, std::size_t N>
int Tree<K, V, N>::balance_node(int node_index) {
    update_height(node_index);
    int balance = get_balance(node_index);

    if (balance > 1) {
        if (get_balance(storage[node_index].left) < 0) {
            storage[node_index].left =
rotate_right(storage[node_index].left);
        }
        return rotate_right(node_index);
    }

    if (balance < -1) {
        if (get_balance(storage[node_index].right) > 0) {
            storage[node_index].right =
rotate_left(storage[node_index].right);
        }
        return rotate_left(node_index);
    }

    return node_index;
}
```

AVL-дерево поддерживает балансировку с помощью вращений left и right.

Вывод

В ходе лабораторной работы были изучены:

1. Шаблоны в C++ — параметризованные классы и функции
2. Класс Graph — представление графа и алгоритм Прима для MST
3. AVL-дерево — самобалансирующееся дерево поиска
4. Вращения — left и right для балансировки

Шаблоны позволяют создавать универсальный код, работающий с различными типами данных.